

PROJECT PROPOSAL

Database Management System

Group Members:

Amna Ahsan – 24K0755

Manahil Zulfiqar – 24K0675

Laiba Ashfaq Adamji – 24K0840

1. Executive Summary

This project proposes the development of a comprehensive full-stack web application for visualizing and analyzing Formula 1 racing statistics. The system will demonstrate advanced database concepts including hybrid database architecture (PostgreSQL + MongoDB), normalization theory (1NF-5NF), complex SQL operations, and RESTful API design.

The application will serve as both an educational demonstration of database management principles and a practical tool for F1 enthusiasts to explore racing data, driver performance, team statistics, and race analytics through an interactive web interface.

2. Problem Statement

Formula 1 generates massive amounts of structured and unstructured data:

- **Structured data:** Driver information, race results, championship standings, team details
- **Unstructured data:** Real-time telemetry, weather conditions, session logs

Current challenges:

1. **Data complexity:** F1 data spans relational (drivers, teams) and non-relational (telemetry, logs) domains
2. **Query performance:** Complex multi-table joins required for analytics
3. **Real-time requirements:** Telemetry data needs fast ingestion and retrieval
4. **Normalization:** Proper database design to avoid redundancy and maintain integrity

Proposed Solution: A hybrid database architecture combining PostgreSQL (relational data) and MongoDB (NoSQL data) with a React-based visualization frontend.

3. Project Objectives

Primary Objectives:

1. Design and implement a normalized relational database (PostgreSQL) for F1 structured data
2. Integrate a NoSQL database (MongoDB) for semi-structured telemetry and logging data
3. Develop a RESTful API backend using Node.js/Express
4. Create an interactive frontend dashboard with React and modern visualization libraries
5. Demonstrate database concepts: ER modeling, normalization, transactions, views, hybrid queries

Secondary Objectives:

1. Implement ACID transaction processing with isolation levels
2. Create database views for optimized query performance
3. Build responsive, animated UI with modern design patterns
4. Document API endpoints and database schema comprehensively

4. Scope

In Scope:

- Database design and implementation (PostgreSQL + MongoDB)
- Backend API with 20+ endpoints
- Frontend dashboard with 5+ pages
- Data visualization (charts, tables, statistics)
- Transaction management and concurrency control
- Database views for common queries
- Hybrid query service for cross-database operations

Out of Scope:

- Real-time live race tracking
- User authentication and authorization
- Machine learning predictions
- Mobile applications
- Production deployment and scaling

5. Technical Architecture

5.1 Database Layer

PostgreSQL (Relational):

- **Strong Entities:** Driver, Team, Circuit, Season
- **Weak Entities:** Race (depends on Season + Circuit), PracticeSession (depends on Race)
- **Ternary Relationships:**
 - RaceParticipation (Driver + Team + Race)
 - TeamDriverSeason (Team + Driver + Season)
- **Normalization:** 1NF → 5NF implementation
- **Views:** 5 materialized views for common analytics queries

MongoDB (NoSQL):

- **Collections:** Telemetry, Weather, SessionLog
- **Flexible schemas** for varying data structures
- **High-frequency data** ingestion support

5.2 Backend Layer

Stack: Node.js, Express.js, Sequelize (ORM), Mongoose (ODM)

API Endpoints (20+ routes):

- [/api/drivers/*](#) - Driver information and statistics
- [/api/races/*](#) - Race results and details
- [/api/teams/*](#) - Team performance metrics
- [/api/views/*](#) - Database view queries
- [/api/hybrid/*](#) - Cross-database hybrid queries

Features:

- Transaction processing with ACID guarantees
- Complex SQL queries (JOINS, aggregations, window functions)
- Manual "JOIN" implementation across PostgreSQL and MongoDB
- Error handling and validation

5.3 Frontend Layer

Stack: React 18, Vite, TailwindCSS, Recharts, React Router, Framer Motion

Pages:

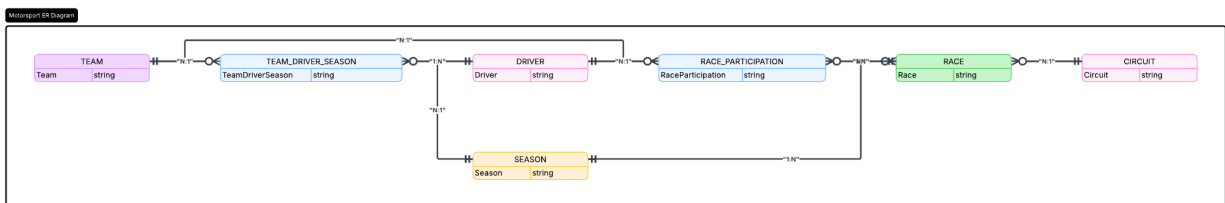
1. Dashboard - Overview statistics and charts
2. Drivers - Driver performance and standings
3. Teams - Team statistics and lineups
4. Races - Race calendar and results
5. Standings - Championship tables

Design Features:

- Dark gradient theme with F1 branding colors
- Glassmorphism effects
- Animated transitions and interactions
- Responsive grid layouts
- Interactive data visualizations

6. Database Design

6.1 ER Model



6.2 Normalization Strategy

1NF: Atomic values, no repeating groups **2NF:** No partial dependencies on composite keys **3NF:** No transitive dependencies **BCNF:** Every determinant is a candidate key **4NF:** No multivalued dependencies **5NF:** Lossless join decomposition

6.3 Key Database Concepts Demonstrated

- Entity types (strong vs. weak)
- Relationship degrees (binary, ternary)
- Referential integrity constraints
- Composite and foreign keys
- Database views (virtual tables)
- Indexing strategies
- Transaction isolation levels

7. Implementation Plan

Phase 1: Database Setup (Week 1-2)

- Design ER diagrams
- Implement PostgreSQL schema
- Create MongoDB collections
- Seed sample F1 data
- Create database views

Phase 2: Backend Development (Week 3-4)

- Set up Express server
- Implement Sequelize models
- Create Mongoose schemas
- Develop API controllers
- Build hybrid query service
- Implement transaction utilities

Phase 3: Frontend Development (Week 5-6)

- Initialize React project with Vite
- Create routing structure
- Build reusable components
- Implement data fetching
- Add charts and visualizations
- Apply styling and animations

Phase 4: Testing & Documentation (Week 7-8)

- Test API endpoints
- Verify database operations
- Write comprehensive documentation
- Create deployment guides
- Prepare demonstration materials

8. Technology Stack

Layer	Technology	Purpose
Frontend	React 18	UI framework
	Vite	Build tool
	TailwindCSS	Styling
	Recharts	Data visualization
	Framer Motion	Animations
Backend	Node.js + Express	Server runtime
	Sequelize	PostgreSQL ORM
	Mongoose	MongoDB ODM
Databases	PostgreSQL	Relational data
	MongoDB	NoSQL data
Development	Git/GitHub	Version control
	Nodemon	Auto-restart
	dotenv	Environment config

9. Expected Deliverables

1. **Source Code**
 - Complete backend codebase
 - Complete frontend codebase
 - Database schema files
 - Seed scripts
2. **Documentation**
 - README.md with setup instructions
 - API documentation
 - Database schema documentation
 - Deployment guide

3. **Demonstration**

- Working application (local or deployed)
- Database populated with sample data
- Interactive dashboard

4. **Reports**

- Project proposal (this document)
- Final project report
- Technical documentation

10. **Success Criteria**

The project will be considered successful if:

- Database normalized to at least 3NF
- Minimum 20 API endpoints functional
- Frontend displays data from both databases
- Transactions maintain ACID properties
- Database views provide performance optimization
- Hybrid queries successfully JOIN across databases
- UI is responsive and visually appealing
- Documentation is comprehensive and clear

11. **Risks and Mitigation**

Risk	Impact	Mitigation
Database schema changes	High	Version control, migration scripts
API performance issues	Medium	Indexing, query optimization, caching
Frontend-backend sync errors	Medium	TypeScript, API contracts, error handling
Time constraints	High	Prioritize core features, MVP approach
Learning curve (new tech)	Medium	Start early, online resources, documentation

12. **Timeline**

Total Duration: 8 weeks

- **Weeks 1-2:** Database design and implementation
- **Weeks 3-4:** Backend API development
- **Weeks 5-6:** Frontend development
- **Weeks 7-8:** Testing, documentation, polish

13. Conclusion

This project will deliver a comprehensive demonstration of database management principles through a real-world F1 analytics application. By combining relational and NoSQL databases, the system showcases modern hybrid architecture approaches while maintaining strong theoretical foundations in normalization, transaction processing, and query optimization.

The resulting application will serve as both an educational tool for understanding database concepts and a practical platform for exploring Formula 1 statistics.